

use of experience sharing. The final algorithm is identical to independent A2C (Algorithm 19) but uses Equation 9.89 for learning the parameters.

Why would one consider using experience sharing over parameter sharing? Experience sharing is certainly more expensive in terms of computational power per environment step, as it increases the size of the batch and has a number of neural network parameters that scale with the number of agents. However, it has been shown (Christianos, Schäfer, and Albrecht 2020) that initializing agent-specific neural networks can lead to learning better policies: not making the assumption that an environment contains strongly homogeneous agents and not restricting policies to be identical can lead to higher converged returns.

In contrast to parameter sharing, experience sharing does not assume that the optimal joint policy $\pi^* = (\pi_1, \pi_2, \dots, \pi_n)$ consists of identical policies $\pi_1 = \pi_2 = \dots = \pi_n$. In addition, the benefit of experience sharing over not sharing any experience or parameters is that it increases the sample-efficiency of algorithms by using all the available trajectories for learning. But another, less apparent, benefit is that it ensures a uniform learning progression between agents. Agents learning from the experience of others can quickly catch up on policies that achieve higher returns, since each agent learns using the experiences from all agents, including agents with more advanced policies. In turn, when agents have a similar learning progression, they get more opportunities to explore actions that require coordination, resulting in better efficiency in data collection.

9.8 Policy Self-Play in Zero-Sum Games

In this section, we will turn our attention to zero-sum games with two agents and fully observable states and actions, in particular turn-taking board games such as chess, shogi, backgammon, and Go. Such games are characterized by three primary aspects, which in combination can make for a very challenging decision problem for an agent:

Sparse reward The game terminates after a finite number of moves, at which point the winning player receives a +1 reward and the losing player receives a -1 reward; or both players receive a 0 reward in the case of a draw outcome. In all other cases there is no reward signal, meaning the reward is always 0 in non-terminal states, until a terminal state is reached.

Large action space The agents can typically choose from a large number of actions, such as moving the many available pieces on the board. This leads to a large branching factor in the search space.

Long horizon Reaching a terminal state (i.e., win/lose/draw) can require tens or even hundreds of moves in some games. Therefore, the agents may have to explore long sequences of actions before they receive any reward.

We can think of such games as a tree, in which each node represents a game state, and each outgoing edge from a node represents a possible action choice for the player whose turn it is in the node. A terminal state is represented by a leaf node that has no outgoing edges. The above aspects mean that the tree can be both very wide (many actions leading to large branching factor) and very deep (many actions needed to reach a terminal state), and that only the leaf nodes can give a non-zero reward. With a limited compute budget, it is typically infeasible to fully explore such complex game trees.

A standard approach to tackle such games is to use heuristic search algorithms such as alpha-beta minimax search, which expands a search tree from each encountered state of the game to compute an optimal action for that state. Such algorithms rely heavily on specialized evaluation functions that are used to guide the search, as well as many other domain-specific adaptations (Levy and Newborn 1982; Campbell, Hoane Jr., and Hsu 2002). This makes such algorithms highly specialized for a specific game, and difficult to adapt to other games. Moreover, the heuristic design choices in such algorithms can limit their achievable performance, for example, due to inaccuracies in heuristic evaluations of game states.

Monte Carlo tree search (MCTS) is a sampling-based method that, similar to alpha-beta search, expands a search tree from the game state but does not require knowledge of specialized evaluation functions (however, if available, such knowledge can also be used in MCTS to further improve its performance). MCTS algorithms grow the search tree by generating a number of simulations from the game state, where each simulation is produced by sampling actions based on information contained in the current search tree. MCTS is essentially a reinforcement learning method in that it can use the same action selection mechanisms and temporal-difference learning operators. However, while reinforcement learning algorithms aim to learn complete policies that choose optimal actions in each state, the focus in MCTS is to compute optimal actions for the *current* state of the game rather than complete policies.

Algorithms that use MCTS in combination with policy self-play and deep learning have achieved “super-human” performance in several games, including chess, shogi, and Go (Silver et al. 2016; Silver et al. 2017; Silver et al. 2018; Schrittwieser et al. 2020). These algorithms use a self-play approach (specifically, “policy self-play” as described in Section 5.5.1) whereby an agent’s

policy is trained against itself. In this section, we will describe one such algorithm called AlphaZero (Silver et al. 2018). The section begins by describing a general MCTS algorithm for Markov decision processes, based on which we will introduce policy self-play in MCTS for zero-sum turn-taking games with fully observable states and actions. The AlphaZero algorithm uses this self-play MCTS algorithm in combination with deep learning to learn an effective evaluation function for the game.

9.8.1 Monte Carlo Tree Search

The pseudocode for a general MCTS algorithm in a Markov decision process is shown in Algorithm 25. In each state s^t , the algorithm performs k simulations¹⁷ $\hat{s}^\tau, \hat{a}^\tau, \hat{r}^\tau, \hat{s}^{\tau+1}, \hat{a}^{\tau+1}, \hat{r}^{\tau+1}, \hat{s}^{\tau+2}, \hat{a}^{\tau+2}, \hat{r}^{\tau+2}, \dots$ (where k is a parameter of the MCTS algorithm) by sampling actions and growing the search tree. We use the notation $\hat{s}^\tau$, $\hat{a}^\tau$, and $\hat{r}^\tau$ to refer to the states, actions, and rewards in a simulation, respectively, starting with state $\hat{s}^\tau = s^t$ and time $\tau = t$. To simplify the pseudocode, we implicitly assume that $\hat{s}^\tau$ refers to both a state and a corresponding node in the search tree; hence, we do not introduce explicit notation to represent trees and nodes. The algorithm can either use the state transition function $\mathcal{T}$, if this is known, or it can use a simulation model $\hat{\mathcal{T}}$ as described in Section 3.6. Note that, rather than building a new tree in each state s^t , MCTS continually uses and grows the tree across all episodes and time steps.

Each node in the search tree carries certain statistics used by the algorithm. In particular, each node $\hat{s}^\tau$ contains a counter $N(\hat{s}^\tau, \hat{a})$ that counts the number of times that action $\hat{a}$ was tried in state $\hat{s}^\tau$, and an action-value function $Q(\hat{s}^\tau, \hat{a})$ to estimate the value (i.e., expected return) of each action in the state. When a new node $\hat{s}^\tau$ is created and added to the tree, the function *InitializeNode*($\hat{s}^\tau$) sets the counter and action-value function to zero, that is, $N(\hat{s}^\tau, \hat{a}) = 0$ and $Q(\hat{s}^\tau, \hat{a}) = 0$ for all $\hat{a} \in A$.

To generate a simulation, the function *ExploreAction*($\hat{s}^\tau$) returns an action to be tried in each visited state $\hat{s}^\tau$ in the simulation. A basic way to sample actions is ϵ -greedy action selection, such as used in Algorithm 3 (page 35). Another, often used action selection method in MCTS is to first try each action once, and then deterministically choose an action that has the highest *upper confidence*

17. Other terms used to refer to simulations in MCTS include “rollouts” and “playouts.” These terms come from the original research focus of using MCTS in competitive board games. We prefer the neutral term “simulation” since MCTS can be applied in a broad range of decision problems.

Algorithm 25 Monte Carlo tree search (MCTS) for MDPs

```

1: Repeat for every episode:
2:   for  $t = 0, 1, 2, 3, \dots$  do
3:     Observe current state  $s^t$ 
4:     for  $k$  simulations do
5:        $\tau \leftarrow t$ 
6:        $\hat{s}^\tau \leftarrow s^t$  ▷ Perform simulation
7:       while  $\hat{s}^\tau$  is non-terminal and  $\hat{s}^\tau$ -node exists in tree do
8:          $\hat{a}^\tau \leftarrow \text{ExploreAction}(\hat{s}^\tau)$ 
9:          $\hat{s}^{\tau+1} \sim \mathcal{T}(\cdot | \hat{s}^\tau, \hat{a}^\tau)$ 
10:         $\hat{r}^\tau \leftarrow \mathcal{R}(\hat{s}^\tau, \hat{a}^\tau, \hat{s}^{\tau+1})$ 
11:         $\tau \leftarrow \tau + 1$ 
12:       if  $\hat{s}^\tau$ -node does not exist in tree then
13:          $\text{InitializeNode}(\hat{s}^\tau)$  ▷ Expand tree
14:       while  $\tau > t$  do ▷ Backpropagate
15:          $\tau \leftarrow \tau - 1$ 
16:          $\text{Update}(Q, \hat{s}^\tau, \hat{a}^\tau)$ 
17:       Select action  $a^t$  for state  $s^t$ :
18:        $\pi^t \leftarrow \text{BestAction}(s^t)$ 
19:        $a^t \sim \pi^t$ 

```

bound (UCB),¹⁸ formally

$$\hat{a}^\tau = \begin{cases} \hat{a} & \text{if } N(\hat{s}^\tau, \hat{a}) = 0 \\ \arg \max_{\hat{a} \in A} \left(Q(\hat{s}^\tau, \hat{a}) + \sqrt{\frac{2 \ln N(\hat{s}^\tau)}{N(\hat{s}^\tau, \hat{a})}} \right) & \text{otherwise} \end{cases} \quad (9.90)$$

where $N(\hat{s}) = \sum_{\hat{a}} N(\hat{s}, \hat{a})$ is the number of times state $\hat{s}$ has been visited. (If multiple actions $\hat{a}$ have $N(\hat{s}^\tau, \hat{a}) = 0$, then we can choose them in any order.) UCB assumes that the rewards $\hat{r}^\tau$ lie in the normalized range $[0, 1]$ (Auer, Cesa-Bianchi, and Fischer 2002). UCB action selection tends to be more effective than ϵ -greedy action selection if only a relatively small number of actions can be tried in a state. Moreover, the finite-sample estimation errors of UCB are bounded when used in MCTS (Kocsis and Szepesvári 2006).

The simulation stops once a state $\hat{s}^l$ has been reached that is a leaf node in the search tree, meaning the state $\hat{s}^l$ has never been visited and no corresponding

18. This version of UCB was originally called “UCB1” in the work of Auer, Cesa-Bianchi, and Fischer (2002).

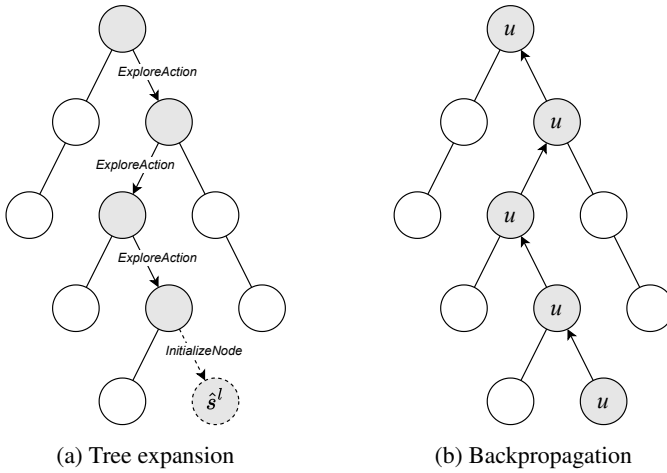


Figure 9.25: Tree expansion and backpropagation in MCTS. Each node in the tree corresponds to a state. The tree is expanded by sampling actions (edges) until a previously unvisited state $\hat{s}^l$ is reached, and a new node for the state is initialized and added to the tree (dashed circle). The new state $\hat{s}^l$ is evaluated via $u = f(\hat{s}^l)$, and u is propagated back through the predecessor nodes in the tree until reaching the root node (e.g., using Equation 9.91).

node exists in the search tree. The search tree is then expanded by using $InitializeNode(\hat{s}^l)$ to initialize a new node corresponding to $\hat{s}^l$. $InitializeNode(\hat{s}^l)$ can use an evaluation function $f(\hat{s}^l)$ to get an initial estimate of the value of the state $\hat{s}^l$. Such a value estimate could be obtained in different ways. If domain knowledge is available, then a heuristic function f could be manually created to compute value estimates based on the domain knowledge. For instance, sophisticated evaluation functions have been created for the game of chess based on vast amounts of expert knowledge (Levy and Newborn 1982; Campbell, Hoane Jr., and Hsu 2002). More generally, a domain-agnostic approach could be to sample actions uniform-randomly until a terminal state is reached, but such an approach can be highly inefficient.

Once the new state $\hat{s}^l$ has been initialized in the search tree and a value estimate $u = f(\hat{s}^l)$ is obtained, MCTS backpropagates u and the rewards $\hat{r}^\tau$ through the nodes that have been visited in the simulation, starting with the predecessor node $\hat{s}^{l-1}$ and following its predecessors until reaching the root node. For each node $\hat{s}^\tau$ to be updated, the function $Update(Q, \hat{s}^\tau, \hat{a}^\tau)$ increases the counter $N(\hat{s}^\tau, \hat{a}^\tau) = N(\hat{s}^\tau, \hat{a}^\tau) + 1$, and the action-value function Q is updated. In general, Q may be updated using any of the known temporal-difference

learning rules used in RL (Section 2.6). For example, we may update $Q(\hat{s}^\tau, \hat{a}^\tau)$ using the off-policy Q-learning update rule with learning target $\hat{r}^\tau + \gamma u$ if $\tau = l - 1$, and learning target $\hat{r}^\tau + \gamma \max_{a' \in A} Q(\hat{s}^{\tau+1}, a')$ if $\tau < l - 1$. However, if the MDP is terminating (i.e., each episode will terminate at some point) and all rewards are zero until a terminal state is reached, such as in the zero-sum board games considered in this section, then u can be backpropagated directly via¹⁹

$$Q(\hat{s}^\tau, \hat{a}^\tau) \leftarrow Q(\hat{s}^\tau, \hat{a}^\tau) + \frac{1}{N(\hat{s}^\tau, \hat{a}^\tau)} [u - Q(\hat{s}^\tau, \hat{a}^\tau)] \quad (9.91)$$

which computes the average value of u for state-action pair $\hat{s}^\tau, \hat{a}^\tau$. Note that, in this case, if $\hat{s}$ is a terminal state giving reward $\hat{r}$ (such as +1 for win, -1 for lose), then the evaluation $u = f(\hat{s})$ should be equal or close to the reward $\hat{r}$. Figure 9.25 depicts the process of expanding the search tree and backpropagating u through the predecessor nodes.

After the k simulations have been completed, MCTS uses *BestAction*(s^t) to select an action a^t for the current state s^t , based on the information stored in the node corresponding to s^t . In general, *BestAction*(s^t) returns a probability distribution π^t over actions $a \in A$. A common choice is to select a most-visited action, $a^t \in \arg \max_a N(s^t, a)$, or an action with the highest value, $a^t \in \arg \max_a Q(s^t, a)$. After executing a^t and observing the new state s^{t+1} , the MCTS procedure repeats and k simulations are performed in s^{t+1} , and so forth.

9.8.2 Self-Play MCTS

The idea of policy self-play is to train an agent's policy against itself, meaning the same policy is used to choose actions for each agent. This requires that the agents in the game have symmetrical roles and egocentric observations, such that the same policy can be used from the perspective of each agent. In zero-sum board games, the focus of this section, agents have symmetric roles since they are direct opponents (each agent is trying to defeat the other agent) and they generally have access to the same types of actions.²⁰ In contrast, if the agents have non-symmetrical roles in the game, such as offense and defense players in a football team, then using the same policy for each agent does not make sense since the different roles require different actions.

19. Equation 9.91 additionally assumes an undiscounted return objective.

20. The self-play approach we describe in this section can also work even if the agents' roles are not completely symmetric. For example, the game of chess is not perfectly symmetric because the white player makes the first move, which gives it a slight advantage; and the relative starting positions of the king and queen differ between the two players. What matters is that we can define egocentric observations and agents have access to the same actions, and that there are generalizable characteristics over the agents' roles. For example, in chess, many play strategies can be used by both white and black players.

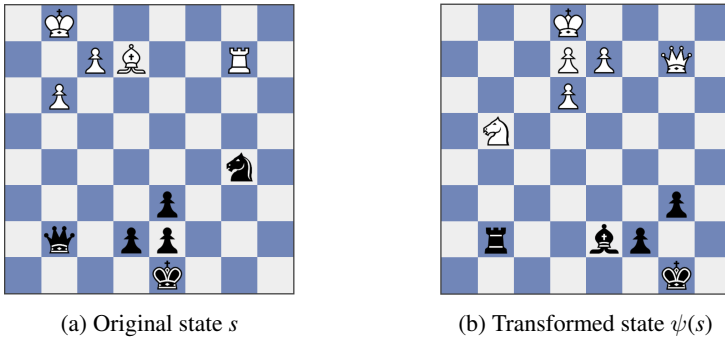


Figure 9.26: State transformation in chess. The state s is transformed to $\psi(s)$ by rotating the board by 180 degrees and swapping the color of the pieces.

An agent's observation is egocentric if the information contained in the observation is relative to the agent. In the chess example, in which agents observe the full state of the game, the state may be represented as a vector $s = (i, x, y)$ where i is the number of the agent whose turn it is, x is a vector containing the locations of agent i 's pieces (set to -1 if the piece was removed), and y is a similar vector for the opponent's pieces. Suppose agent 1 controls the white pieces and its policy π_1 is conditioned on this state vector. Therefore, π_1 will assume that the information about white pieces is located in the x -part of the state vector. To use agent 1's policy π_1 to control the black pieces for agent 2, we can transform the state vector $s = (2, x, y)$ (in which x stores the locations of the black pieces, and y for white pieces) by changing the agent number and swapping the order of the x/y vectors, $\psi(s) = (1, y, x)$. If we visualize the state s , as shown in Figure 9.26, where white pieces are located at the top of the board and black pieces are at the bottom, then the transformation $\psi(s)$ corresponds to rotating the board by 180 degrees and swapping the colors of the pieces from black to white and vice versa. The transformed state $\psi(s)$ can now be used by agent 1's policy to choose an action for agent 2, that is, $a_2 \sim \pi_1(\cdot \mid \psi(s))$.

Given such a state transformation method, we can adapt the general MCTS algorithm shown in Algorithm 25 to implement a policy self-play approach for zero-sum games in which the agents take turns. Essentially, from the perspective of agent 1, in self-play the MCTS simulations become a Markov decision process in which agent 1 chooses the action in each time step. Using $\hat{s}_t^i$ to denote the state and agent i whose turn it is to choose an action in that

state, the self-play simulation process for agent 1 becomes:

$$\pi_1(\hat{s}_1^\tau) \rightarrow \hat{a}^\tau \rightarrow \pi_1(\psi(\hat{s}_2^{\tau+1})) \rightarrow \hat{a}^{\tau+1} \rightarrow \pi_1(\hat{s}_1^{\tau+2}) \rightarrow \hat{a}^{\tau+2} \rightarrow \pi_1(\psi(\hat{s}_2^{\tau+3})) \rightarrow \hat{a}^{\tau+3} \dots \quad (9.92)$$

To implement this process in the MCTS simulations, we modify the algorithm to replace $\hat{s}^\tau$ with $\hat{s}_i^\tau$ to keep track of which agent is choosing the action in the state, and in the function calls of *ExploreAction* (Algorithm 25), *InitializeNode* (Algorithm 25), and *Update* (Algorithm 25) we input $\hat{s}_i^\tau$ if $i = 1$, and $\psi(\hat{s}_i^\tau)$ if $i \neq 1$. Additionally, since the evaluation $u = f(\hat{s}^l)$ is always from the perspective of agent 1 (e.g. +1 reward if agent 1 wins), in function *Update* when using Equation 9.91 to update $Q(\hat{s}_i^\tau, \hat{a}^\tau)$, we have to flip the sign of u if $i \neq 1$. Thus, these functions always operate on states as if it was agent 1's turn in each state. Note that MCTS controls agent 1, hence the time steps t in Algorithm 25 include only the times in which it is agent 1's turn.

The above version of self-play MCTS uses the *current* version of the policy to select actions for each agent. This algorithm can be further modified by training the policy not just against the current version of itself but also against *past* versions of the policy. For example, we may maintain a set Π that contains copies of agent 1's policies (or the Q functions from which they are derived) from different points of the training process. Using Π , the self-play MCTS algorithm may sample any policy from this set when selecting actions for agent 2. In this way, we may obtain a potentially more robust policy for agent 1 by forcing it to perform well against not just one specific policy (itself) but also against past versions of the policy. Thus, Π serves a similar purpose to the replay buffer $\mathcal{D}$ (Section 8.1.3), which is to reduce overfitting. We will explore this idea in more depth in Section 9.9.

9.8.3 Self-Play MCTS with Deep Neural Networks: AlphaZero

AlphaZero (Silver et al. 2018) is based on MCTS with self-play against the current policy, as described in the previous sections. Additionally, it uses a deep convolutional neural network parameterized by θ to learn an evaluation function

$$(u, p) = f(s; \theta) \quad (9.93)$$

which, for any input state s , predicts two elements:

- u : The expected outcome of the game from state s , $u \approx \mathbb{E}[z | s]$, where z is either +1 for a win, -1 for a loss, and 0 for a draw. Thus, u estimates the (undiscounted) value of state s .
- p : A vector of action selection probabilities in state s , with entries $p_a = \Pr(a | s)$ for each action $a \in A$, to predict the action probabilities computed by MCTS in the *BestAction* function for state s .

AlphaZero learns these value estimates and action probabilities entirely from self-play, starting with randomly initialized parameters θ and using stochastic gradient descent to minimize a combined loss function

$$\mathcal{L}(\theta) = (z - u)^2 - \pi^\top \log p + c \|\theta\|^2 \quad (9.94)$$

with parameter c to control the weight of the squared L2-norm, $\|\theta\|^2 = \theta^\top \theta$, which acts as a regulariser to reduce overfitting. This loss is computed based on data $D = \{(s^t, \pi^t, z^T)\}$, where for each episode in the MCTS algorithm (Algorithm 25), s^t is the state at time t (Algorithm 25), π^t is the action probability distribution for state s^t computed in *BestAction* (Algorithm 25), and z^T is the game outcome in the last time step T of the episode. As usual, a mini-batch of this data is sampled when computing the loss in stochastic gradient descent. Thus, $f(s)$ learns value estimates u to predict the outcome z , and action probabilities p to predict the MCTS action probabilities π , for any given state s .

The action probability vector p is used in AlphaZero to guide the action selection in the MCTS simulations. When expanding the search tree for new node $\hat{s}^l$ (Algorithm 25), the function *InitializeNode*($\hat{s}^l$) computes $(u, p) = f(\hat{s}^l; \theta)$ with the current parameters θ and, in addition to setting $N(\hat{s}^l, \hat{a}) = 0$ and $Q(\hat{s}^l, \hat{a}) = 0$, initializes action prior probabilities $P(\hat{s}^l, \hat{a}) = p_{\hat{a}}$ for all $\hat{a} \in A$. The function *ExploreAction* (Algorithm 25) uses N, Q, P in a formula similar to the UCB (Equation 9.90) method

$$\hat{a}^\tau = \begin{cases} \hat{a} & \text{if } N(\hat{s}^\tau, \hat{a}) = 0 \\ \arg \max_{\hat{a} \in A} \left(Q(\hat{s}^\tau, \hat{a}) + C(\hat{s}^\tau) P(\hat{s}^\tau, \hat{a}) \frac{\sqrt{N(\hat{s}^\tau)}}{1 + N(\hat{s}^\tau, \hat{a})} \right) & \text{otherwise} \end{cases} \quad (9.95)$$

where $C(\hat{s})$ is an additional exploration rate. Therefore, AlphaZero explores actions similarly to UCB, but biases the action selection based on the predicted probabilities p . After completing the MCTS simulations from the root node s^t , AlphaZero uses a function *BestAction*(s^t) that selects an action a^t either proportionally (for exploration/training) with respect to the root visit counts $N(s^t, \cdot)$, that is, $a^t \sim \frac{N(s^t, \cdot)}{\sum_{a \in A} N(s^t, a)}$, or greedily $a^t \in \arg \max_a N(s^t, a)$.

The complete specification of AlphaZero includes various additional implementation details about the network representation of f , the exploration rate C , the addition of exploration noise in action probabilities, the sampling of data batches to compute the loss, action masking, and so on. These details can be found in the supplementary material of Silver et al. (2018). However, the core idea behind AlphaZero is to learn an effective evaluation function f purely from self-play MCTS, and to use this function to guide the tree search in a vast search space in order to find optimal actions.

AlphaZero (playing white) vs.	Win	Draw	Loss
Stockfish (chess)	29.0%	70.6%	0.4%
Elmo (shogi)	84.2%	2.2%	13.6%
AlphaGo Zero (Go)	68.9%	–	31.1%

Figure 9.27: AlphaZero match results reported by Silver et al. (2018). Results show percentage of won/drawn/lost games for AlphaZero when playing white.

Silver et al. (2018) reported results for AlphaZero in the games of chess, shogi, and Go, in which it was matched against several strong game-playing programs: “Stockfish” for chess, “Elmo” for shogi, and AlphaGo Zero (Silver et al. 2017) for Go trained for three days. In each game, AlphaZero performed $k = 800$ MCTS simulations in each state s^t . Separate instances of AlphaZero were trained for nine hours (fourty-four million games) in chess, twelve hours (twenty-four million games) in shogi, and thirteen days (140 million games) in Go. Based on Elo rating (Elo 1960), AlphaZero first outperformed Stockfish after four hours, Elmo after two hours, and AlphaGo Zero after thirty hours. Figure 9.27 shows the reported match results for the final trained instances of AlphaZero (when playing white) in the three games. AlphaZero was able to achieve this performance using the same general self-play MCTS method in each game, without requiring heuristic evaluation functions based on expert knowledge.²¹ While Stockfish and Elmo evaluated approximately sixty million and twenty-five million states per second, respectively, AlphaZero evaluated only about sixty thousand states per second in chess and shogi. AlphaZero compensated for the lower number of evaluations by using its deep neural network to focus more selectively on promising actions.

9.9 Population-Based Training

Policy self-play, as presented in Section 9.8, requires that the agents in the game have symmetrical roles and egocentric observations, such that the same policy can be used from the perspective of each agent. Specifically, in Section 9.8 we focussed on symmetrical zero-sum games with two agents. Can we generalize policy self-play to general-sum games with two or more agents?

21. However, AlphaZero *did* use domain knowledge in the construction of the input and output features of f (e.g., legality of castling, repetition count for current position, underpromotions for pawn moves, piece drops in shogi, and so on).

We already discussed one type of generalization of policy self-play at the end of Section 9.8.2, in which an agent’s policy is trained not just against itself, but also against a distribution over *past* versions of its policy. In doing so, we can potentially obtain a more robust policy by training it against a diverse *population* of policies, thereby reducing overfitting to itself. The term “population” is used in this context because the number of policies in the population can grow over successive generations (also called “epochs”), and the policies may change and adapt in each generation to perform better within the population.

Population-based training generalizes this idea to work in general-sum games with two or more agents, in which the agents may have different (non-symmetric) roles, actions, and observations. Essentially, the idea is to maintain multiple policy populations Π_i^k , one for each agent $i \in I$ in the game, and to grow and modify each population with respect to the other populations over successive generations $k = 1, 2, 3, \dots$. The general steps in population-based training can be summarized as follows:

Initialize populations: Create an initial policy population Π_i^1 for each agent i . For example, each population may start with a single uniform policy that selects actions uniform-randomly. If domain knowledge or recorded data of past interactions are available, then other types of policies could be created for the initial populations.

Evaluate populations: In generation k , evaluate the current policies $\pi_i \in \Pi_i^k$ in each agent’s population to measure the performance of each policy (e.g., based on expected returns or Elo ranking) with respect to the policies in the other populations. For example, this can be done by running episodes with different combinations of policies from the populations.

Modify populations: Based on the evaluations of policies $\pi_i \in \Pi_i^k$ in each population, modify the existing policies and/or add new policies to the populations. For example, existing policies could be modified by making random changes to their parameters, or by copying the parameters of better-performing policies in the same population (Jaderberg et al. 2017). New policies could be created by training new policies against some distribution over the policies in other populations, or by creating new policies that optimize a type of best-response diversity (Rahman, Fosong, et al. 2023).

Once the populations have been modified, we have a new generation of populations Π_i^{k+1} for each agent i and the process repeats by reevaluating the new populations, followed by further modifications to the policies, and so on. The process terminates after a predetermined number of generations, or once

some other termination criterion is achieved such as reaching a certain average performance of policies in each population.

Variants of this population-based training approach have been successfully applied in diverse complex games (e.g., Lanctot et al. 2017; Jaderberg et al. 2019; Liu et al. 2019; Vinyals et al. 2019). In the next subsection, we will describe a general variant of population-based training called policy space response oracles (PSRO) (Lanctot et al. 2017), which can directly incorporate the notion of best-response policies and game-theoretic solution concepts from Chapter 4 as sub-routines. PSRO can be applied to optimize policies in general-sum games with two or more agents and full or partial observability. We will then discuss a complex MARL algorithm called AlphaStar (Vinyals et al. 2019) which, among other components, uses population-based training and was the first algorithm to reach top-level performance in the full game of *StarCraft II*.

9.9.1 Policy Space Response Oracles

Policy space response oracles (PSRO) (Lanctot et al. 2017), itself based on the double oracle algorithm (McMahan, Gordon, and Blum 2003), refers to a family of population-based training algorithms that learn policies in general-sum games with full or partial observability and two or more agents (e.g., a POSG or any other game model defined in Chapter 3). PSRO builds on empirical game-theoretic analysis (e.g., Wellman 2006), which constructs a *meta-game* as an abstraction of the underlying game G , and applies equilibrium analysis to the meta-game. A meta-game is a finite normal-form game in which the possible actions of the agents correspond to specific policies π_i in the underlying game G and the reward functions $\mathcal{R}_i(\pi_1, \dots, \pi_n)$ of the meta-game give the average return for agent i in the game G if the agents select policies $\pi_1, \dots, \pi_n$. The important benefit of meta-games is that they allow for tractable approximate equilibrium computation by limiting the sets of possible policies the agents can choose in G .

PSRO constructs such meta-games in each generation of the population-based training to evaluate and grow the policy populations. The pseudocode of PSRO is given in Algorithm 26, and the main steps are shown in Figure 9.28. We detail the steps in the following paragraphs.

PSRO starts by initializing the populations Π_i^1 , for each agent i , to contain one or more randomly generated policies. Then, in each generation k , PSRO constructs a meta-game (i.e., normal-form game) M^k in which the action space of each agent is set as the current population of the agent, that is, $A_i = \Pi_i^k$. The reward $\mathcal{R}_i(\pi_1, \dots, \pi_n)$ for each agent i when agents use policies $\pi_1, \dots, \pi_n$ in the meta-game M^k is estimated empirically, by running one or more episodes of the underlying game G using the policies $\pi_1, \dots, \pi_n$ and averaging agent

Algorithm 26 Policy space response oracles (PSRO)

-
- 1: Initialize populations Π_i^1 for all $i \in I$ (e.g., random policies)
 - 2: **for** each generation $k = 1, 2, 3, \dots$ **do**
 - 3: Construct meta-game M^k from current populations $\{\Pi_i^k\}_{i \in I}$
 - 4: Use meta-solver on M^k to obtain distributions $\{\delta_i^k\}_{i \in I}$
 - 5: **for** each agent $i \in I$ **do** $\triangleright$ Train best-response policies
 - 6: **for** each episode $e = 1, 2, 3, \dots$ **do**
 - 7: Sample policies for other agents $\pi_{-i} \sim \delta_{-i}^k$
 - 8: Use single-agent RL to train π'_i wrt. π_{-i} in underlying game G
 - 9: Grow population $\Pi_i^{k+1} \leftarrow \Pi_i^k \cup \{\pi'_i\}$
-

i 's returns from these episodes. Thus, in the limit of running infinitely many episodes, the reward will converge to the expected return of the agent, that is, $\mathcal{R}_i(\pi_1, \dots, \pi_n) = U_i(\pi_1, \dots, \pi_n)$. The construction of the meta-game M^k completes the population evaluation step.

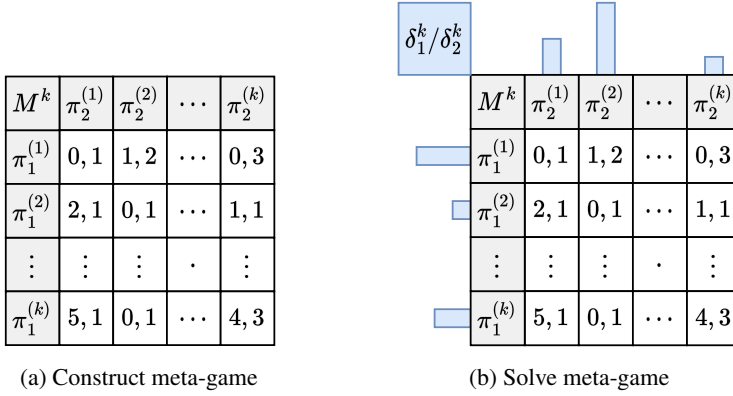
Given the meta-game M^k , PSRO uses a so-called “meta-solver” to compute probability distributions δ_i^k for each population Π_i^k , where $\delta_i^k(\pi_i)$ is the probability assigned to policy $\pi_i \in \Pi_i^k$. For example, the meta-solver may compute any of the solution concepts discussed in Chapter 4, such as Nash equilibrium.²² To avoid situations in which δ_i^k concentrate their probability masses on a small number of policies, which could potentially lead to overfitting of policies learned by PSRO, the original PSRO method enforces a lower bound $\delta_i^k(\pi_i) > \epsilon$ for each policy $\pi_i \in \Pi_i^k$, where $\epsilon > 0$ is a parameter (Lanctot et al. 2017). Other types of distributions may be computed based on the requirements of the game, such as in AlphaStar discussed in Section 9.9.3.

Based on M^k and the population distributions δ_i^k , PSRO uses an *oracle* to compute a new policy π'_i to add to each agent's population. The standard oracle used in PSRO computes a best-response policy in the game G with respect to the distribution $\delta_{-i}^k(\pi_{-i}) = \prod_{j \neq i} \delta_j^k(\pi_j)$:

$$\pi'_i \in \arg \max_{\pi_i} \mathbb{E}_{\pi_{-i} \sim \delta_{-i}^k} [U_i(\langle \pi_i, \pi_{-i} \rangle)] \quad (9.96)$$

Such a best-response policy π'_i can be obtained by using a single-agent RL algorithm to train π'_i over multiple episodes in the underlying game G , where in each episode the policies of the other agents $j \neq i$ are sampled as $\pi_j \sim \delta_j^k$. Note

22. With some small modifications, it is also possible to use a correlated equilibrium meta-solver to compute a joint distribution δ^k over the joint-policy space $\Pi^k = \Pi_1^k \times \dots \times \Pi_n^k$, where $\delta^k(\pi)$ is the probability assigned to join policy $\pi \in \Pi^k$ (Marris et al. 2021).



M^k	$\pi_2^{(1)}$	$\pi_2^{(2)}$	$\dots$	$\pi_2^{(k)}$	π_2'
$\pi_1^{(1)}$	0, 1	1, 2	$\dots$	0, 3	?
$\pi_1^{(2)}$	2, 1	0, 1	$\dots$	1, 1	?
$\vdots$	$\vdots$	$\vdots$	$\cdot$	$\vdots$	?
$\pi_1^{(k)}$	5, 1	0, 1	$\dots$	4, 3	?
π_1'	?	?	?	?	?

(c) Add new policies via oracle

Figure 9.28: Steps in PSRO for a two-agent game, at generation k . (a) Construct a meta-game M^k based on the current policy populations $\{\Pi_i^k\}_{i \in I}$. The cells show the estimated average returns for each agent (we show example values for illustration purposes) when they use the respective policies from the row and column. (b) Use meta-solver on M^k to obtain distributions δ_i^k for each population (e.g., Nash equilibrium). The shown bars on the left and top represent probabilities assigned by δ_i^k to the respective policies in the population. (c) Use the oracle to compute a new policy (e.g., best-response policy) π_i' for each agent i with respect to the other populations Π_{-i}^k and distributions δ_{-i}^k , and add π_i' to the population Π_i^k . PSRO then repeats these steps, where step (a) estimates values for the missing entries for the new policies π_i' (marked by “?”)

that, in this case, the policy π'_i may only be an *approximate* best response if the RL training does not converge in the given training budget, or if it converges to a local optimum. The computation of best-response policies π'_i (Algorithm 26 of Algorithm 26) can be done in parallel for each agent $i \in I$.

Once we have the new best-response policies π'_i for each agent i , PSRO adds the policies to the respective populations to obtain the next generation, i.e.

$$\Pi_i^{k+1} = \Pi_i^k \cup \{\pi'_i\}. \quad (9.97)$$

PSRO then repeats the above steps, constructing a new meta-game M^{k+1} to reevaluate the populations (i.e., computing the new entries in $\mathcal{R}_i$ for the newly added policies), meta-solving M^{k+1} and adding further policies to the populations, and so on. This process terminates after a predefined number of generations, or if the new best-response policies π'_i are already contained in their respective populations Π_i^k for each i , in which case the process has reached a fixed point.

The basic version of PSRO presented here can be computationally expensive and difficult to scale to many agents. Constructing a meta-game M^k involves sampling potentially many episodes in the underlying game for each joint policy $\pi \in \Pi_1^k \times \dots \times \Pi_n^k$, and this space of joint policies grows geometrically with the size of the populations. Solving M^k via Nash equilibrium also has exponential complexity (see Section 4.11) and the added problem of non-uniqueness (see Sections 4.7 and 5.4.2). Finally, using RL to compute best-response policies can also be costly depending on the complexity of the underlying game. To address these computational bottlenecks, a number of improvements for PSRO have been developed that make the method more computation-efficient and scalable (e.g., Lanctot et al. 2017; Balduzzi et al. 2019; McAleer et al. 2020; Muller et al. 2020; Smith, Anthony, and Wellman 2021). In Section 9.9.3, we will see how the algorithm AlphaStar builds on PSRO ideas to learn strong policies in a very complex, high-dimensional multi-agent game.

9.9.2 Convergence of PSRO

If PSRO uses a meta-solver that computes *exact* Nash equilibria for a meta-game, and an oracle that computes *exact* best-response policies in the underlying game G , then the distributions $\{\delta_i^k\}_{i \in I}$ converge to a Nash equilibrium of G . To see why this happens, we can consider two questions: Is PSRO always guaranteed to converge? And when it does converge, is the outcome guaranteed to be a Nash equilibrium?

We assume a finite underlying game G (i.e., with finite sets of agents, actions, states, and observations) that terminates after a finite number of time steps. The first thing to note is that, for any distribution δ_{-i}^k for agents $-i$, there will

always be a *deterministic* best-response policy π'_i for agent i . This is because, by fixing the policies of other agents according to $\pi_{-i} \sim \delta_{-i}^k$, the underlying game G reduces to a finite MDP if G is a stochastic game, or to a finite POMDP if G is a POSG. In both cases, we know (see Chapter 2) that there always exist deterministic optimal policies π'_i for the MDP/POMDP. While there may also exist stochastic (i.e., not deterministic) optimal policies, these need not be considered by the oracle, since any stochastic policy can be obtained by a probabilistic mixture of deterministic policies, which can be achieved via the distributions δ_i^k (we will see an example of this below). Furthermore, since G has finite episodes, there are a finite number of deterministic policies that can be enumerated for G .

All of the above means that there is a finite set of deterministic best-response policies $\pi'_i \in \Pi'_i$ for each agent i in G that the oracle can select from in each generation k . In the worst case, PSRO will need to add all best-response policies from Π'_i to the population Π_i^k for each agent i before it terminates (recall that PSRO terminates in generation k if the oracle's selected policies π'_i are already contained in the populations Π_i^k for each agent i , respectively). Therefore, returning to the first question above, we know that PSRO will eventually converge. In other games it may be possible for PSRO to converge much earlier, by reaching a point where the new best-response policies π'_i selected by the oracles are already contained in the respective agent populations Π_i^k . At that point, by the definition of the Nash equilibrium (Section 4.4), we know that no agent can unilaterally select a best-response policy $\pi'_i \in \Pi'_i$ that is not already contained in Π_i^k in order to improve its expected returns against the distribution δ_{-i}^k . Therefore, returning to the second question above, we know that if PSRO converges then the outcome $\{\delta_i^k\}_{i \in I}$ must be a Nash equilibrium in the underlying game G .

To illustrate these points, we give two examples in non-repeated matrix games with two agents (the reward matrices are shown in Figure 3.2, page 46). As a convenient shorthand, we will say “action X” to refer to the deterministic policy that assigns probability 1 to action X.

First, consider the non-repeated Rock-Paper-Scissors game. Suppose PSRO initializes the populations so that agent 1's initial population Π_1^1 contains action R, and agent 2's initial population Π_2^1 contains action P. Figure 9.29 shows the populations Π_i^k , distributions δ_i^k , and best responses π'_i for both agents over generations $k = 1, \dots, 5$. It can be seen that PSRO converges to the unique Nash equilibrium of the game, in which both agents randomize uniformly. However, since this equilibrium assigns positive probability to all joint actions, PSRO will need to add all deterministic policies (i.e., actions) in both populations before it converges. At generation $k = 5$, in which both populations contain all actions,

k	Π_1^k	Π_2^k	δ_1^k	δ_2^k	π_1'	π_2'
1	<u>R</u>	<u>P</u>	1	1	S	P
2	R, <u>S</u>	P	(0, 1)	1	S	R
3	R,S	<u>R</u> ,P	$(\frac{2}{3}, \frac{1}{3})$	$(\frac{2}{3}, \frac{1}{3})$	P	R/P
4	R, <u>P</u> ,S	R,P	$(0, \frac{2}{3}, \frac{1}{3})$	$(\frac{1}{3}, \frac{2}{3})$	R	S
5	R,P,S	R,P, <u>S</u>	$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$	$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$	R/P/S	R/P/S

Figure 9.29: PSRO in the non-repeated Rock-Paper-Scissors matrix game. The table shows the populations Π_i^k , distributions δ_i^k , and best responses π_i' for both agents over generations $k = 1, \dots, 5$. New entries in populations are shown underlined. At $k = 3$, R/P for π_2' means that both actions are best responses and the oracle can select one of them (and same at $k = 5$ for R/P/S). PSRO converges to the unique Nash equilibrium in which both agents randomize uniformly.

any best responses π_1', π_2' will already be contained in the populations Π_1^5, Π_2^5 , respectively, and hence PSRO terminates.

Next, consider the non-repeated Prisoner's Dilemma game. Suppose PSRO initializes the populations Π_i^1 of both agents to contain the defect action. Since the populations contain only single policies, the distributions δ_i^1 will assign probability 1 to these policies. The oracle will then return the defect action as the only best-response π_i' for each agent, which is already contained in both populations. Hence, in this example, PSRO converges to the unique Nash equilibrium of the game (in which both agents defect) without enumerating all possible deterministic policies.

Note that the above assumes *exact* meta-games M^k , in which the meta-rewards are equal to the expected returns of the agents, that is, $\mathcal{R}_i(\pi_1, \dots, \pi_n) = U_i(\pi_1, \dots, \pi_n)$ for all $i \in I$ and all joint policies $(\pi_1, \dots, \pi_n)$ from the populations $\Pi_1^k, \dots, \Pi_n^k$. It also assumes *exact* best-responses π_i' for each agent. Both assumptions will likely not hold in practice, since M^k is constructed using a finite number of sampled game episodes under the different joint policies,²³ and since the best-response policies π_i' are learned using a RL algorithm.

For other combinations of meta-solvers and oracles, PSRO will evolve different policy populations and may converge to different solution types (e.g., Muller et al. 2020; Marris et al. 2021).

23. Theoretical bounds exist on the number of sampled episodes required such that a Nash equilibrium in the estimated meta-game is also a (approximate) Nash equilibrium in the exact meta-game (Tuyls et al. 2020).

9.9.3 Grandmaster Level in *StarCraft II*: AlphaStar

StarCraft II is a popular real-time strategy game in which two or more players collect resources and build armies consisting of diverse units (e.g., infantry, tanks, aircraft) in order to defeat each other in battle. *StarCraft II* shares all the difficulties outlined at the beginning of Section 9.8: players can choose from a very large set of possible actions, and each match can involve many thousands of sequential actions before a terminal state (win or lose) is reached. In addition, players in *StarCraft II* only have a limited view of the environment: they only see what the units in their army see within their limited view regions. Moreover, the three different races available in *StarCraft II* (Terran, Protos, Zerg) offer different units and require different play strategies. These aspects make *StarCraft II* a highly complex game, and its popularity has led to a large and active community of professional human players that compete in international electronic sports events. In the year 2019, *AlphaStar* (Vinyals et al. 2019) was the first artificial agent to reach Grandmaster level in the full game of *StarCraft II*,²⁴ placing it above 99.8 percent of officially ranked human players according to the ranking metric used in *StarCraft II*. AlphaStar achieved this performance by using a combination of RL and population-based training.

At its core, AlphaStar trains a policy $\pi(a_i^t | h_i^t, z; \theta_i)$ for each race parameterized by θ_i , which assigns probabilities to actions a_i^t at time t , given the history of observations and actions $h_i^t = (o_i^0, a_i^0, o_i^1, a_i^1, \dots, o_i^t)$, and a vector z which summarizes a strategy based on human data (detailed below). An observation o_i^t contains an overview map of the environment (akin to the minimap observed by human players) and a list of visible friendly and enemy units with associated attributes (e.g., remaining health points). An action a_i^t specifies the action type (e.g., move, build, attack), what unit to issue the action to, the target of the action (e.g., where to move), and when the agent wants to select its next action. AlphaStar receives a reward of +1 for winning, -1 for losing, and 0 for a draw outcome; and 0 rewards at all other times. No discounting is used in the learning objective, to reflect the true goal of winning the game.

As human players are limited by physical constraints when playing the game, various constraints were put in place for AlphaStar in order to ensure a fair match against humans. For example, agents were limited to executing at most twenty-two non-duplicate actions in a five second time window.

The action representation used in AlphaStar results in approximately 10^{26} possible actions an agent can choose from in a time step. Exploring such a

24. The *StarCraft* Multi-Agent Challenge (SMAC) (Samvelyan et al. 2019) provides a smaller version of *StarCraft II* and has been used widely in MARL research; see Section 11.3.3.

vast search space from scratch is intractable, especially since the only non-zero rewards are obtained in terminal states after many thousands of sequential actions. Therefore, AlphaStar uses human play data to initialize the policies. Prior to any RL training, each policy is trained via supervised learning to imitate human actions based on recorded matches played by human players. From each match replay, a statistic z is extracted which encodes information about the human player's strategy, such as the order of constructed buildings and statistics about units during the match. The policy is then trained to predict the human action a_i^t given only the history h^t from the match or also z . By extracting a diverse set of statistics z from the recorded matches and conditioning the policy on z , the policy is able to produce diverse strategies.

Following policy initialization, AlphaStar uses a range of deep learning and RL techniques based on A2C (Section 8.2.4) to train a policy against a distribution of opponent agents (i.e., the oracle in PSRO). The policy is either conditioned on a statistic z and the agent receives a reward for following the strategy corresponding to z ; or it is only conditioned on the history h^t and free to choose its own actions. In both cases, the agent is penalized when the action probabilities deviate from the initial (supervised) policy. While the details of the RL techniques are important for the training performance of AlphaStar, for the purpose of this section, we will focus on the population-based training component used in AlphaStar. We refer to the original publication (Vinyals et al. 2019) for details on the used deep learning architectures and RL methods.

AlphaStar uses a type of population-based training called “League training,” which follows a similar approach to PSRO. AlphaStar maintains a single league (population) Π^k of policies corresponding to three different types of agents for each race: main agents, main exploiter agents, and league exploiter agents. The three types of agents differ in the distributions δ_i^k of opponents from the league against which they are trained, at which points in training their current policies (i.e., parameters) are added to the league, and when their policies are reset to the initial parameters obtained from the supervised learning stage.

Let Π^k be the current league at generation k , which contains past policy copies of the three agent types from different points in the training. For a given policy $\pi_i' \in \Pi^k$, AlphaStar uses a method called *prioritized fictitious self-play* (PFSP) to compute a distribution δ_i^k over policies π_i in a subset of Π^k against which to train π_i' , defined as

$$\delta_i^k(\pi_i) \propto f(\Pr[\pi_i' \text{ wins against } \pi_i]) \quad (9.98)$$

where $f: [0, 1] \rightarrow [0, \infty)$ is a weighting function. AlphaStar uses two types of weighting functions:

- $f_{hard}(x) = (1 - x)^p$, where $p \in \mathbb{R}^+$ is a parameter, leads PFSP to focus on the most difficult opponent policies for π'_i . This is the default choice for f .
- $f_{var}(x) = x(1 - x)$ leads PFSP to focus on opponent policies that are at a similar level of performance as π'_i .

The probability $\Pr[\pi'_i \text{ wins against } \pi_i]$ is estimated empirically by running multiple matches of π'_i versus π_i and recording the average win rate for π'_i .

Based on the above, the details of the three agent types used in the league are as follows:

- Main agents (one per race) are trained with a proportion of: 35 percent in self-play, in which the currently learning policy of the main agent is matched against itself (as described in Section 9.8.2); 50 percent over the past policies of all agent types in Π^k using probabilities given by PFSP; and another 15 percent against past policies of main exploiters with probabilities given by PFSP. The policies of main agents are frozen and added to the league every 2×10^9 training time steps. Main agents are never reset to the initial parameters.
- Main exploiter agents (one per race) are trained against the main agents to exploit their weaknesses. With a proportion of 50 percent, or if their current estimated probability of winning is less than 0.20, exploiter agents are trained against past main agent policies in the league with probabilities given by PFSP using weighting function f_{var} . The other 50 percent of training is done against the currently learning policies of the main agents. Main exploiter agents are added to the league when they manage to defeat all three main agents in at least 70 percent of the played matches, or after 4×10^9 training time steps. Their parameters are then reset to the initial parameters.
- League exploiter agents (two per race) are trained against all of the policies contained in the league, to identify strategies that no policy in the league is effective against. They are trained against policies in Π^k using probabilities given by PFSP, and they are added to the league if they manage to defeat all policies in the league in at least 70 percent of the played matches, or after 2×10^9 training time steps. At that point, with a probability of 0.25, their parameters are reset to the initial parameters.

For the matches against top-level human players reported by Vinyals et al. (2019), AlphaStar initialized the policies via supervised learning based on a publicly available dataset of 971,000 anonymised matches with human players that ranked in the top 22 percent on the *StarCraft II* metric “Match Making Rating” (MMR), which is similar to the Elo ranking used in chess (see Section 9.8.3). After forty-four days of League training using thirty-two third-generation tensor processing units (TPUs) (Jouppi et al. 2017), the final

trained main agents corresponding to the three races were evaluated in matches against human players (under an anonymous account) via the official online match-making system used in *StarCraft II*. The main agents were evaluated without conditioning on the statistics z in the policy. The final main agents achieved MMR ratings that placed them above 99.8 percent of officially ranked human players, and at “Grandmaster” level for all three races. Interestingly, the initial policies after supervised learning already achieved a fairly strong performance, ranking them above 84 percent of human players. Vinyals et al. (2019) showed that without using the human play data to initialize policies, the performance of AlphaStar degrades very substantially as the search space becomes too difficult to explore from scratch. It is worth noting that, similarly to AlphaZero (Section 9.8.3), AlphaStar was able to achieve this superior performance using general deep learning and RL methods. These methods could similarly be applied to other multi-agent decision problems that require complex planning and interactions over long timescales.

9.10 Summary

This chapter has presented a range of “deep” MARL algorithms that build on deep learning methods to learn policies in complex environments. The main concepts are summarized as follows:

- In deep MARL, different training and execution paradigms have played a significant role in how algorithms are developed and used. The concept of *centralized information* includes information that may be shared across agents, such as parameters, gradients, observations, actions, or anything else that is not typically included in an agent’s observation space. Three paradigms are discussed: In *centralized training and execution*, agents learn and operate by sharing centralized information throughout both training and execution phases. In *decentralized training and execution*, agents are isolated, neither sharing information nor communicating with each other, ensuring fully independent learning based solely on their local observations. Lastly, *centralized training and decentralized execution (CTDE)* represents a hybrid and popular approach. Agents utilize centralized information to facilitate learning during the training phase. However, agents learn policies that are only conditioned on the local observations, thus enabling a fully independent execution of their policies.
- *Independent learning* is a class of MARL algorithms in which agents independently use single-agent RL to learn their policies. This class of algorithms

in deep MARL can sometimes reach similar performance to more complicated methods despite its relative simplicity. Independent learning algorithms can be utilized with the decentralized training and execution paradigm.

- The first extension of independent learning we discuss are MARL algorithms based on the *multi-agent policy gradient*. These algorithms typically learn a centralized value function to consider that the returns an agent receives are also dependent on the policies of other agents. We introduced *centralized critics* $V(h_i^t, z^t; \theta_i)$, which are not only conditioned on an agent's observation history but also centralized information z^t such as the observations of other agents. The same methodology can be extended to *centralized action-value critics* by learning a action value $Q(h_i^t, z^t, a^t; \theta_i)$, which is also conditioned on the joint action.
- *Value decomposition algorithms* are designed to address environments (i.e., games) with common rewards. These algorithms attempt to find the contribution of each individual agent to a common reward signal. This problem is also known as the multi-agent credit assignment problem and it is commonly solved by attempting to factorize the joint-action value functions by assuming that it can be approximated using simpler functions, for example, $Q(s, \langle a_1, a_2, a_3 \rangle) \approx Q(s, \langle a_1, a_2 \rangle) + Q(s, \langle a_1, a_3 \rangle)$. *Value decomposition networks* (VDN) is such an algorithm that assumes that $Q(s, \langle a_1, a_2, a_3, \dots \rangle) \approx Q(a_1) + Q(a_2) + Q(a_3) + \dots$, and thus learning a value functions for each agent that sum up to the environment's expected common returns. Similarly, *QMIX* approximates the joint action as a linear combination (albeit with monotonic requirements) of the individual action-values.
- *Agent modeling* is concerned with learning models of other agents to inform the policy or value functions of the modeling agent. For example, an agent model can be trained to reconstruct the policies of other agents. By representing such agent models as neural networks, the agent models can further generalize to unseen states. Instead of reconstructing the policies of other agents, agent models can also learn compact representations of the policies of other agents. By condition policies and value functions on such compact representations, agents can learn to act with respect to the policies of other agents and more efficiently learn to adapt to their policies.
- Deep MARL algorithms may be able to benefit from *experience* or *parameter sharing*. This methodology was inspired after observing that in some environments, the optimal joint policy π^* is composed of (nearly) identical individual policies, that is, $\pi_1^* = \pi_2^* = \pi_3^* = \dots$. Parameter sharing works in such environments, by using shared parameters for the policy or value functions, and substantially improves both the efficiency of gradient descent and limits

the search space. Similarly experience sharing consists of sharing trajectories collected by the agents between them, in order to train independent policies with more a more diverse batch of experience.

- Chess, Go, and other zero-sum games with particularly large state spaces have inspired a class of *policy self-play* algorithms that train a single policy by letting it play against itself. *AlphaZero* is a self-play algorithm that uses Monte Carlo tree search (MCTS) in each encountered state, a sampling-based tree search algorithm that can look ahead to find the optimal move for the current state. AlphaZero was able to learn champion-level policies in diverse zero-sum games, including chess, shogi, and Go.
- *Population-based training* generalizes the idea of self-play to general-sum games with two or more agents and full or partial observability. These algorithms maintain populations (i.e., sets of) of policies for each agent, and evolve these populations over multiple generations by evaluating and modifying the policies in the populations. *Policy space response oracles* (PSRO) is a population-based training algorithm that generates meta-games in each generation, which are normal-form games in which the actions for each agent correspond to the policies in their populations, and the rewards for a joint action (i.e., combination of policies) correspond to the agents' expected returns when using these policies in the underlying game. PSRO uses game-theoretic solution concepts such as Nash equilibrium to compute distributions over the agents' policies in their populations, based on which an oracle function computes new policies for each agent (normally best-response policies with respect to the distributions) and adds those policies to the agents' respective populations. *AlphaStar* is an algorithm that operates like PSRO (with some modifications) and was able to achieve Grandmaster level in the game of *StarCraft II*.

In this chapter, we have discussed various deep MARL algorithms that use deep learning techniques to learn policies in high-dimensional, partially observable multi-agent environments. We have covered important concepts and strategies used in these algorithms to tackle challenges such as partial observability, non-stationarity, multi-agent credit assignment, and equilibrium selection. Next, we move from theory to practice. In the upcoming chapter, we will show how some of these MARL concepts can be implemented in practice.